Petro Ladkin

Product Engineer. I take products from idea - or from stuck - to production. Robotics, backend, mobile; now with AI agents.

Chernivtsi, Ukraine | petro.ladkin@gmail.com | petroladkin.dev | linkedin.com/in/petroladkin | github.com/petroladkin

SUMMARY

First program in 1995; paid to write software since 2006. I take a product at any stage - an idea, an MVP, a stalled project, an empty repo, a domain I have never seen - to production. Today I own intent, specs, acceptance and release, and direct AI agents to build and review what is needed.

SKILLS

Now	Product engineering / ROS 1+2 / C++ / Go / .NET (C#) / AI agents (specs, acceptance, agent review)
Also	Python / embedded (STM32) / iOS, macOS / security and VoIP / code generation / small teams
Earlier	Windows Mobile / Windows desktop (WinAPI, WTL) / Unity, AR

EXPERIENCE

Deus Robotics | Remote

Product Engineer — Apr 2026 - present

I own intent, specs, acceptance and release; AI agents build and review to that.

- Moved my own engineering to AI agents: in October 2026, 94% of my commits are agent co-authored. Built a multi-model code-review command - 6 specialist agents plus an arbiter - used in four repos.
- Rewrote the fleet operator web UI in React in 4 calendar days with a team of AI agents, replacing a 73.7k-line Flutter app; parity checked against 237 items of the old UI.
- Task dispatcher for a client delivery-robot project, sole author: first working version 9 days after the first commit, a full service with 1,097 tests in about 6.5 weeks (Go, design-first OpenAPI, PostgreSQL).
- Fail-closed face anonymisation of every uploaded photo: from about 440 to 40 ms per photo; runtime image cut from 486 to 260 MB.
- Still maintainer and release owner of the fleet-management API (.NET): its unit-test suite reached 2,271 test methods.
- Co-maintain the Go core of a cloud AI shelf-inspection platform for third-party robots: half of all commits, 8 releases; introduced its quality gates (coverage ratchet, linters, vulnerability scanners, race detector).
- Wrote the agent rules and reference implementation for the company's AI-native process pilot.

Go / .NET (C#) / React / PostgreSQL / OpenAPI / AI agents

Senior Software Engineer — Oct 2021 - Apr 2026

Robot software in ROS 2, from first prototypes to the fleet backend.

- Learned ROS, which I had never seen, and had a layout-marking robot MVP driving on the map in about two months, navigating by a total station.
- Co-led the migration of the robot software from ROS 1 to ROS 2 without stopping the fleet: built the foundation (bridge proof of concept, multi-repo workspace, Docker, CI) and was the primary author of 10 of the 16 robot-side components.
- Built from an empty repo the adapter between ROS 2 and the company's own robots and chargers - a C++20 TCP server speaking their binary protocol - and took it to release 1.0 in ten months and 2.0 the next year; 358 test cases.
- Robot driver for wheels, MCU and conveyor (7.9k lines of C++, 70% mine): a generator from the ODrive endpoint description instead of hand-written register maps; an async rewrite that fixed crashes in 5 nodes.
- Became maintainer and release owner of the fleet-management API (.NET microservices): migrated it to .NET 8 and grew its unit-test suite from 7 to 1,753 test methods.
- Created the Go core of a cloud AI shelf-inspection platform for third-party robots.
- Code review: reviewed 408 MRs of other engineers; authored 562 (516 merged).

- Layout robot, sole developer (152 commits in 8 months): total-station localisation by circle intersection, heading from a short driven vector, prism angle-error correction, a Unity operator console over ROS-TCP with A* path planning.
- CANopen control app for a new 2-wheel platform with BLDC drives: 3.7k lines of Python in 22 days.
- Quality and releases for the ROS 2 repos: standardised about ten packages in three weeks (linter, a README per node, first unit tests); release branches with pinned dependency manifests; merge gate in 2023.

ROS 1+2 / C++ / Python / C# / Go / CAN / CANopen / Modbus / ODrive / Docker / GitLab CI

Staff-Eye GmbH | Remote 2021-2022, part-time

Senior Software Development Engineer

Projects for the company's clients, one at a time. **Refactored a client's production C# test scripts without access to their runner: built a harness that recorded every output against mocked hardware calls, then refactored against that record - within the one-month deadline; the client came back with the next stalled project.**

- OMG IDL code generator to C, C++, C#, Python, FlatBuffers and Protobuf: learned Flex and Bison from scratch; version 1.0.0 in four months with 119 unit tests and a manual for every language.
- Restarted an STM32 firmware project that had stood still for a year and ported it from the F100/F103 family to F303: every peripheral driver from SPL to HAL, CubeIDE alongside Keil. My first real embedded project.

C++ / C / C# / Flex / Bison / FlatBuffers / Protobuf / STM32 HAL

Freelance (AR agency) | Remote 2021

Software Development Engineer

Three apps for an AR agency, one after another. **Live History: an AR guide along the Great Amber Route - at each of 10 locations a character from that era appears, anchored in place, and tells its story with lip-synced speech; built alone from scratch to release.**

- Kaup: an app about the reconstructed Viking village Kaup; the agency came with the concept only, I built it alone from scratch to release.
- Kaup 3D: a virtual tour of the village without AR - the app, the animations and characters going about daily tasks. All three published to the App Store and Google Play.

Unity / C# / AR / LipSync / Lean Localization

CleverBooks | Remote 2021

Software Development Engineer

Educational AR apps. **Space AR: the latest source code had been lost with the previous developer; rebuilt the app from a version two or three releases older up to what was live, then shipped new content.**

- Blocks AR: built from early MVP code - an AR mode and a Tetris engine adapted to AR space.
- Moved Space AR from Vuforia to EasyAR; both apps published to the App Store, Google Play and Amazon.

Unity / C# / AR Foundation / Vuforia / EasyAR

RC Car in AR - own product 2020-2021

Software Engineer, Designer, SMM

My first Unity project; idea, code, design and marketing, alone. **Two months from the first commit to the App Store and Google Play.**

- All code (AR Foundation, my own car physics, human occlusion, 3D audio), localization into 9 languages with my own framework, in-app purchases and ads, promo videos, ad campaigns.
- 59 commits, about 4k lines of my C#; releases 1.0 to 1.4 in January-May 2021.

Unity / C# / AR Foundation / Unity Services

Auxoft LLC | Chernivtsi, Ukraine / remote 2015-2020

Co-founder & CTO

KeyReel: a Bluetooth password manager - credentials stay encrypted on the phone and are sent to login pages when the phone is near the computer. **Pitched the startup in one call with a start budget I wrote myself; the condition was a proof of concept in 2 months - I built it, and the MVP got funded.**

- Took KeyReel from a 4-month MVP to a product on five platforms: iOS, macOS, Windows, browser extensions and Android.
- As CTO owned every technical decision - languages, frameworks, process; wrote 96% of the 68.5k-line Apple apps and 5,798 of the company's 8,917 commits.

- Came up with probabilistic login-form recognition; in an internal benchmark KeyReel auto-filled 488 of 500 sites (indicative: tuned on the same list).
- Built 2FA and U2F into KeyReel: an authenticator inside the password database, and the phone acting as a security key through a macOS driver.
- Set up the release process on my own git-flow CLI: two-week sprints, nightly to release builds - about 450 tagged builds.
- Wrote the first cross-browser extension framework: one business-logic codebase that later carried KeyReel to Chrome, Safari, Firefox, Edge, Opera, Brave and Vivaldi.
- Rescued the stalled Android app by hiring and directing a freelancer; it reached Google Play in July 2020. iOS app rated 4.8 from 12 ratings; reviewed by The Mac Observer (2018), launched on ProductHunt.
- Shipped iOS 12 Password AutoFill two months after iOS 12; hired the second engineer; mentored an intern who built test tooling for the form parser.

Objective-C / Swift / C++ / Cocoa / UIKit / CoreBluetooth / IOKit / Thrift / JavaScript

Iosoftware | Chernivtsi, Ukraine / remote 2012-2014, part-time

Software Engineer

iOS apps for a small startup. **As lead developer, took BattleHorn, a castle-defense game, from a one-battlefield MVP to an App Store release in autumn 2013.**

- Came in for small graphics fixes in JukeMaster, an iOS DJ app, and found why scratching failed: the UI and audio threads blocked each other. Rewrote that part; the gig became a direct contract.
- Added Wi-Fi multiplayer and a single-player mode against AI with RakNet; owned App Store publishing.

Objective-C / C++ / UIKit / AudioQueue / Cocos2d-x / RakNet

Freelance on oDesk | Chernivtsi, Ukraine / remote 2008-2012, part-time

Software Engineer

Windows Mobile apps and games, alongside the full-time job. **Took over PhoneyCall mid-development - a custom, undocumented UI framework over raw WinAPI, no access to its author - and shipped it in 137 hours.**

- The client came back with a larger contract: four more products in 624 hours, including a full UI rewrite of the game "Done In 50 Seconds".
- Client rating 4.4 of 5 on both rated contracts - with almost no English at the time.

C++ / C# / WinAPI / WTL / .NET CF / TAPI / MAPI

SecurStar GmbH | Chernivtsi, Ukraine 2006-2015

Junior to Senior Software Engineer, System Architect, Team Lead

Encrypted calls for mobile phones. **When our main customers' network dropped the old protocol, built a new P2P-based product in 4 months; customers moved over.**

- As a junior, rewrote SecurePhone from C# to C++ and shipped it.
- PhoneCrypt: MVP, authentication and media servers (NAT traversal), Windows Mobile and iOS clients; the iOS core was still in production ~9 years after I left.
- Team Lead of 6 engineers (3 senior); 2 of my 3 juniors grew to middle.
- Release automation; moved the team from SourceSafe to SVN to Git.
- Wrote the first PhoneCrypt iOS client in about 2 months with no prior Apple-ecosystem experience.
- Built a BlackBerry prototype in about 1 month; voice latency kept growing on Java ME, so the decision not to ship was made on a working prototype, not on opinion.
- Wrote a layer over WTL whose layout rules adapt at runtime to any Windows Mobile screen size.
- As Team Lead designed and supervised the Windows desktop client, SIP gateway, server admin panel and P2P protocol auto-tester; administered the company's Git servers.

C / C++ / Objective-C / C# / PHP / WinAPI / WTL / UIKit / SIP / Speex / AMR

EDUCATION, PATENT, LANGUAGES

Education	Engineer's degree in automation, 1999-2004, Ivano-Frankivsk National Technical University of Oil and Gas
Patent	Co-inventor of US patent 10,764,056 B2 "Short-distance network electronic authentication" (2020)
Languages	Ukrainian (native), English (B1-B2)

EARLIER TOOLS

UI	WTL / wxWidgets / Qt / Windows Forms / WinAPI / MFC
Platforms	Android / Windows Mobile / BlackBerry
Security	Encryption (voice, text, HDD, flash) / OpenSSL
Databases	PostgreSQL / MySQL
Web	HTML / CSS / SASS / JavaScript / jQuery / Jade